

LLM-ERP 系統操作手冊

以大語言模型驅動的智慧企業資源規劃系統

版本：v0.1.0 | 更新日期：2026-05-07

1. 系統概述

LLM-ERP 是一套開源的智慧 ERP 系統，讓使用者透過**自然語言**管理整個工廠流程，不需要記憶 T-code 或點選單。系統涵蓋 7 大模組：

模組	代碼	核心功能
 自然語言介面	—	支援中文與英文，系統自動判斷語言回應
 庫存管理	MM	料號管理、庫存查詢、入出庫異動、儲位追蹤
 採購管理	PP	供應商管理、採購單生命週期、供應商評分
 BOM 工程	ENG	產品結構管理、多階展開、缺料檢查
 生產派工	MFG	工單管理、機台排程、動態重排程 (3 策略)
 品質管理	QM	檢驗單管理、不合格品(NC)追蹤、矯正措施(CAPA)
 會計財務	FI	會計科目、傳票/分錄、AR 逾期管理、月結
 報表生成	—	中英文指令產出 PDF 報表 (庫存/AR/採購/生產/損益)
 戰情室	—	SVG 價值流儀表板、即時事件動畫、跨螢幕顯示

2. 安裝需求

硬體需求

配置	最低	建議
RAM	4 GB	16 GB (含本地 LLM)
CPU	2 cores	8 cores
本地 LLM	—	16 GB RAM + GPU (可選)

軟體需求

- Python 3.11+
- Node.js 18+
- (選用) Docker Desktop — 用 PostgreSQL 正式環境
- (選用) Ollama — 本地 LLM 推理

3. 安裝步驟

3.1 下載專案

```
git clone https://github.com/fanchanyu/llm-erp.git
cd llm-erp
```

3.2 後端設定

```
cd backend
cp .env.example .env
# 編輯 .env，填入你用的 LLM Provider 的 API Key
```

`.env` 檔案說明：

變數	說明	範例
LLM_PROVIDER	LLM 提供商	deepseek / ollama / anthropic / openai
LLM_MODEL	模型名稱	deepseek-chat / gemma4:e4b / claude-sonnet-4
MAX_TOOL_ROUNDS	工具調用輪數	5 (雲端) / 8-10 (本地)
DEEPSEEK_API_KEY	DeepSeek API Key	sk-xxxx

3.3 申請 API Key (選擇一個 Provider)

LLM-ERP 需要一個 LLM (大型語言模型) 來驅動自然語言功能。以下是各家 Provider 的申請方式：

◆ **DeepSeek (推薦, 最便宜)**

1. 前往 https://platform.deepseek.com/sign_up 註冊帳號
2. Email 驗證 → 登入
3. 點選左側「API Keys」→「創建 API Key」
4. 複製 key (格式: sk-xxxxxxxxxxxxxxxxxx)
5. 填入 `.env`: `DEEPSEEK_API_KEY=sk-xxxx`

- **費用**: 約 ¥1 元人民幣可處理 200~500 次查詢
- **模型**: deepseek-chat
- **適合**: 剛開始試用、開發階段

◆ **Anthropic Claude (最聰明)**

1. 前往 <https://console.anthropic.com/> 註冊
2. 登入後點選「API Keys」→「Create Key」
3. 複製 key (格式: sk-ant-xxxxxxxxxxxxxx)
4. 填入 `.env`:
LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-4
ANTHROPIC_API_KEY=sk-ant-xxxx

- **費用**: \$3/M 輸入 + \$15/M 輸出 tokens
- **模型**: claude-sonnet-4 / claude-haiku-3-5 (便宜快速)
- **適合**: 正式生產環境、需要高準確率

◆ **OpenAI GPT (最普及)**

1. 前往 <https://platform.openai.com/signup> 註冊
2. 登入 → 左上角「API Keys」→「Create new secret key」
3. 複製 key (格式: sk-proj-xxxxxxxxxxxxxx)
4. 填入 `.env`:
LLM_PROVIDER=openai

```
LLM_MODEL=gpt-4o
OPENAI_API_KEY=sk-proj-xxxx
```

- 費用：\$2.50/M 輸入 + \$10/M 輸出 (gpt-4o)
- 模型：gpt-4o / gpt-4o-mini (省錢)
- 適合：已有 OpenAI 帳號者、通用場景

♦ **OpenRouter** (一站多用，可選多種模型)

```
1. 前往 https://openrouter.ai/keys 註冊
2. 登入 → 「Create Key」
3. 複製 key (格式: sk-or-v1-xxxxxxxxxx)
4. 填入 .env :
    LLM_PROVIDER=openrouter
    LLM_MODEL=deepseek/deepseek-chat
    OPENROUTER_API_KEY=sk-or-v1-xxxx
```

- 費用：依模型不同 (可用最便宜的)
- 模型清單：<https://openrouter.ai/models>
- 適合：想比較多家模型、切換方便

♦ **Ollama** (本地模型，完全免費，不需 API Key)

如果你的電腦有 16GB+ RAM，可以在本地跑模型，完全不需要 API Key：

```
# 安裝 Ollama
curl -fsSL https://ollama.com/install.sh | sh

# 下載 Gemma4 模型 (8B, 約 9.6GB)
ollama pull gemma4:e4b

# 確認 Ollama 正在執行
curl http://localhost:11434/api/tags

# 設定 .env :
# LLM_PROVIDER=ollama
# LLM_MODEL=gemma4:e4b
# MAX_TOOL_ROUNDS=8
```

- 費用：完全免費，只需要硬體
- 硬體需求：16GB RAM (8B 模型) / 32GB (12B 模型)
- 適合：資料不外洩、不想付 API 費用

⚠ **API Key 安全提醒**：Key 只能放在 `.env`，絕對不能放到 CSV 資料檔、Git 提交、或貼到 GitHub 上。

3.8 啟動後端

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

後端會在 `http://localhost:8000` 啟動。

3.9 啟動前端

```
cd frontend
npm install
npm run dev
```

前端會在 `http://localhost:5173` 啟動。

3.10 啟動本地 LLM (選用)

如果你選擇使用本地模型 (Ollama) , 請先參考 [3.3 節 — Ollama 申請方式](#) 安裝 Ollama 並下載模型, 然後確認 Ollama 正在執行:

```
curl http://localhost:11434/api/tags
```

之後回到 [3.8 節](#) 啟動後端即可。

3.11 資料管理 (Import / Export / Reset)

LLM-ERP 提供標準化的資料匯入匯出工具, 所有資料使用**自然鍵** (料號、供應商名稱、產品編號等) —— 不是 UUID, 所以人類可以直接編輯 CSV。

API Key 永遠不會出現在資料檔中, 只存在於 `.env`。

匯入資料

```
cd backend

# 匯入內建範例資料 (12 張表, 71 筆記錄)
python -m scripts.manage_data import scripts/sample_data/

# 或用自己的 CSV
python -m scripts.manage_data import path/to/my-data.csv

# 或匯入整個目錄 (自動按依賴順序處理)
python -m scripts.manage_data import path/to/data-dir/

# 乾執行 (只驗證, 不改資料)
python -m scripts.manage_data import scripts/sample_data/ --dry-run
```

匯出資料

```
python -m scripts.manage_data export ./backup/
# 輸出: 01-parts.csv ~ 12-ar.csv, 共 12 張表
```

重置資料庫

```
python -m scripts.manage_data reset --force
# 清空所有 22 張表
```

查看 Schema

```
python -m scripts.manage_data schema
# 列出所有實體的欄位定義
```

CSV Schema 速查表

檔案	實體	必要欄位	依賴
01-parts.csv	料號主檔	part_no, name, unit	—
02-suppliers.csv	供應商	name	—
03-products.csv	產品	product_no, name	—
04-work-centers.csv	工作站	name	—
05-accounts.csv	會計科目	account_no, name, type, normal_balance	—
06-inventory.csv	庫存量	part_no, location, quantity	parts
07-bom.csv	BOM 表	product_no, part_no, quantity, level	products, parts
08-purchase-orders.csv	採購單	po_no, supplier_name, item_part_no, item_quantity	suppliers, parts
09-production-orders.csv	工單	order_no, product_no, quantity, due_date	products
10-quality.csv	檢驗單	inspection_no, part_no, quantity	parts, purchase-orders
11-accounting.csv	會計傳票	entry_no, description, entry_date, period, line_account_no, line_debit, line_credit	accounts
12-ar.csv	應收帳款	customer_name, invoice_no, amount, due_date	—

匯入工具會自動依賴順序處理，跳過已存在的記錄（冪等操作）。

```
# 安裝 Ollama
curl -fsSL https://ollama.com/install.sh | sh

# 下載 Gemma4 模型 (8B, 約 9.6GB)
ollama pull gemma4:e4b

# 確認 Ollama 正在執行
curl http://localhost:11434/api/tags

# 將 .env 設為：
# LLM_PROVIDER=ollama
# LLM_MODEL=gemma4:e4b
# MAX_TOOL_ROUNDS=8
```

4. 使用指南

4.1 庫存管理 (Inventory)

你可以問任何關於庫存的問題：

中文查詢	說明
「M6x20 螺絲還有多少庫存？」	查詢特定料號庫存
「幫我看一下全部庫存列表」	列出所有庫存
「傳動件有哪些零件？庫存夠嗎？」	按分類查詢
「查一下馬達類零件的庫存狀況」	依名稱模糊查詢
「入庫 M6x20 螺絲 500 顆」	入庫作業
「發料 M6x20 螺絲 100 顆給工單 WO-001」	出庫/領料

4.2 採購管理 (Purchase)

中文查詢	說明
「供應商有哪些？查一下大明螺絲」	查詢供應商
「幫我開一張採購單，向大明螺絲買 M6x20 200 顆」	建立採購單
「目前的採購單有哪些？」	查詢採購單列表
「PO-2026-0001 的貨到了嗎？」	查詢採購單狀態

4.3 BOM 與物料管理

中文查詢	說明
「CNC-001 用哪些料？」	查詢 BOM 結構
「CNC-001 的 BOM 展開給我看」	BOM 多階展開
「CNC-001 要做 5 台，料夠不夠？」	缺料檢查 (核心功能)

4.4 生產派工 (Dispatch)

中文查詢	說明
「目前有哪些工單？」	查詢工單列表
「幫我釋出工單 WO-20260506-001」	釋出工單
「CNC-01 故障，往後推 30 分鐘」	右移重排程 (Right-Shift)
「CNC-01 故障，換到備用機台」	替代路徑重排程 (Route Change)
「有急單 WO-002，插隊優先處理」	急單插隊 (Expedite)

4.5 品質管理 (Quality)

中文查詢	說明
「新增品檢單 M6x20，批號 LOT-001」	建立檢驗單
「查一下品檢記錄有哪些」	查詢檢驗單列表

「M6x20 有不良品嗎？」	查詢不合格品(NC)
「新增 NC，M6x20 尺寸超差，嚴重度=重大」	建立不良品記錄

4.6 會計財務 (Accounting)

中文查詢	說明
「有哪些逾期應收帳款？」	查詢逾期帳款
「查一下應收帳款報表」	查詢 AR 列表
「會計科目有哪些？」	查詢科目表
「開傳票：借庫存 1000 貸銀行存款 1000」	建立傳票/分錄

4.7 跨模組查詢

中文查詢	說明
「M6x20 螺絲採購單 PO-0001 的貨到了嗎？入庫了？」	採購+庫存
「我要生產 5 台 CNC-001，先檢查料夠不夠」	BOM+採購

5. 戰情室 (War Room)

戰情室是一個全螢幕的 SVG 價值流儀表板，適合工廠現場多螢幕監控。

開啟方式：

```
# 前端啟動後，打開瀏覽器：  
http://localhost:5173/war-room.html
```

戰情室功能：

- 🏗️ 6 階段水平排列：供應商 → 採購 → 庫存 → 派工 → 品質 → 會計
- 🔴 動畫粒子流：綠色代表物料流、黃色代表金流
- 📡 底部事件串流：即時顯示最近 30 筆事件
- 🔔 節點發光：事件觸發時對應階段發光回饋
- ▶ 模擬事件按鈕：展示用自動模擬事件生成
- 🔄 數據 15 秒自動刷新

6. Provider 切換

LLM-ERP 支援 5 家 LLM Provider，可隨時切換：

```
# 編輯 backend/.env  
LLM_PROVIDER=deepseek           # 或 ollama / anthropic / openai / openrouter  
LLM_MODEL=deepseek-chat         # 或 gemma4:e4b / claude-sonnet-4 / gpt-4o  
MAX_TOOL_ROUNDS=5               # 雲端 5，本地 8-10
```

切換原則：

- 雲端 **API** (DeepSeek / Claude / GPT)：MAX_TOOL_ROUNDS=5，需要 API Key
- 本地模型 (Ollama / Gemma4)：MAX_TOOL_ROUNDS=8-10，不需要 API Key
- 雙語支援：所有 Provider 皆支援中英文。用中文問就回中文，用英文問就回英文

- 切換後後端會自動重載 (`uvicorn --reload`)

Provider 效能對比 (30 題 Benchmark)

指標	DeepSeek (雲端)	Gemma4 8B (本地 CPU)
	準確率	90% (27/30)
平均回應	7.7s	8.7s
每題成本	~\$0.002	免費
資料主權	外部 API	完全本地

7. 執行 Benchmark

```
cd /mnt/d/Project/LLM_ERP/evaluation

# 執行 30 題標準測試
python3 run_eval.py

# 詳細輸出
python3 run_eval.py --verbose

# 指定輸出檔
python3 run_eval.py --output my-results.json
```

8. 常見問題

Q: 後端啟動失敗？

A: 確認 `.env` 中有正確的 API Key。確認 `pip install -r requirements.txt` 已執行。

Q: LLM 回應太慢？

A: DeepSeek 平均 7.7s，屬於正常。如果是本地模型（Gemma4），CPU 推理約 8-10s，有 GPU 可降到 1-3s。

Q: 查詢結果不對？

A: 嘗試調整 `MAX_TOOL_ROUNDS`（本地模型需要更多輪數）。也可在 system prompt 中增加明確的禁止規則。

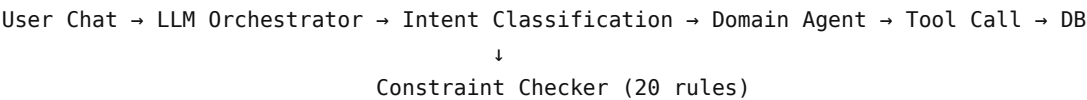
Q: 如何清空資料庫？

A: 使用內建工具：`cd backend && python -m scripts.manage_data reset --force`。或直接刪除 `backend/llm_erp.db` 後重新匯入。

Q: 前端 Vite 抓不到新加的 public 檔案？

A: 重啟 Vite dev server：`Ctrl+C` 後重新 `npm run dev`。

9. 系統架構快速參考



↓
Response + Notifications (Event Bus)

層級	技術	說明
前端	React 18 + TypeScript + Vite + Tailwind	角色儀表板、Chat UI、戰情室
後端	Python FastAPI + SQLAlchemy	7 領域服務、42 API 路由
LLM	DeepSeek / Anthropic / OpenAI / Ollama	27 工具定義、Function Calling
事件	Pub/Sub Event Bus	10 事件類型、角色路由通知
資料庫	SQLite (dev) / PostgreSQL (prod)	22 資料表、Alembic 遷移

10. 典型操作場景 Walkthrough

以下是從接單到出貨的完整流程演練，展示 LLM-ERP 的實際操作方式。

場景一：從訂單到出貨（端到端）

情境：客戶訂了 3 台 **CNC-001** 小型 **CNC** 銑床，**05/15** 前要交貨。

步驟	你做	系統回應
① 查庫存	「CNC-001還有庫存嗎？」	庫存不足，只夠 1 台
② BOM檢查	「CNC-001要做3台， 幫我查料夠不夠」	缺料：軸承缺4顆、 螺絲缺200顆
③ 開採購單	「向大明螺絲買軸承4顆、 向大明螺絲買M6x20螺絲200顆」	採購單 P0-20260507-001 已建立
④ 收貨入庫	「P0-20260507-001貨到了， 軸承入庫4顆，螺絲入庫200顆」	庫存已更新： 軸承：8顆，螺絲：350顆
⑤ 派工生產	「CNC-001做3台，派工生產」	工單 W0-20260507-001 已釋出並排程
⑥ 完工報工	「W0-20260507-001完工了， 3台都做好了」	入庫 3 台 CNC-001 工單完成
⑦ 查成本	「這張工單成本多少？」	材料 \$X，人工 \$Y， 總成本 \$Z

場景二：異常處理 — 機器故障

情境：**CNC-01** 機台故障，正在生產的工單要重新排程。

① 右移重排程（最短時間恢復） 你 → 「CNC-01故障了，把後面的工序往後推2小時」 系統 → 「已將 CNC-01 上的工序全部右移 2 小時，預計完工時間：05/07 16:00」
② 替代路徑重排程（如果有備用機台） 你 → 「CNC-01故障，改到CNC-02生產」 系統 → 「已切換至 CNC-02，工單重新排程完成」
③ 急單插隊（客戶趕貨） 你 → 「客戶急單W0-20260507-001，插隊優先處理」 系統 → 「W0-20260507-001 已排入最前面，預計提前 3 天完成」

場景三：跨模組查詢 — 決策支援

情境：要決定要不要接一張大訂單，需要綜合判斷。

綜合查詢

你 → 「客戶要5台CNC-001、05/20交期，接不接？」

系統 → 「BOM檢查：料夠 → 

產能檢查：CNC-01 05/15前有 40hr 可用 → 

採購交期：軸承交期 3 天 → 05/10 到貨 → 

結論：可以接！預估完工日 05/16」

場景四：品質異常追溯

情境：客戶反應 **CNC-001** 有品質問題，需要追溯。

① 查不良記錄

你 → 「CNC-001最近有不良品記錄嗎？」

系統 → 「找到 1 筆 NC：批號 LOT-B001，軸承尺寸超差」

② 建立 CAPA

你 → 「幫我對 NC-20260505-001 建立改善對策」

系統 → 「CAPA 已建立：更換軸承供應商，增加入庫全檢」

③ 從採購源頭管控

你 → 「查一下這批軸承是哪個供應商？」

系統 → 「大明螺絲，評分已從 4.2 降至 3.5」